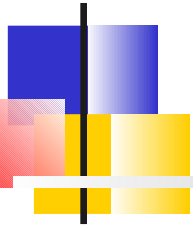


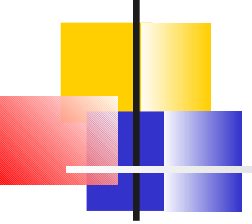
PHP PROGRAMMING BASICS



PHP Programming basics

Lets go through these steps,

- You must be logged in with the admin account to install PHP so login as root
- We need to get installation files from the [PHP Website](#) and save it on your hard drive. You can download it to your download directory where you download all the files from internet or if you have not made one yet then lets create a download directory
`#mkdir /usr/local/download.`



Contd

- Now we will download installation file from [php ftp website](http://ftp.php.net) to our download directory

```
# cd /usr/local/download ftp ftp.php.net
```

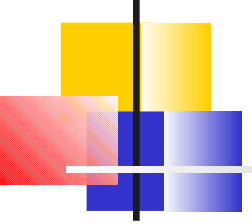
```
# cd pub/distributions
```

```
bin
```

```
get php-5.2.0.tar.gz
```

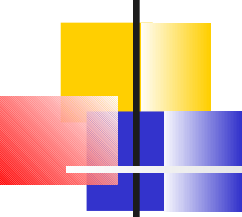
```
bye
```

- The file name may differ according to the PHP version you are downloading. I have downloaded the latest stable release of PHP i.e PHP 5.2.0



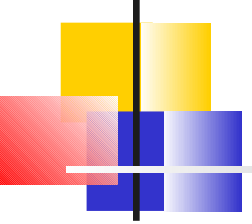
Contd

- now it is acceptable ... the next step is to compile PHP with this command
./configure --with-apache=../httpd --with-config-file-path=/www/conf --enable-track-vars
- now make a PHP executable with
make
- which will take some time and then install it with
make install
- do you think we are done? well, unfortunately no... we need to add php module to apache with this,



Contd

- ```
cd ../httpd
./config.status --activate-
module=src/modules/php5/libphp5.a
then
#make
 then stop apache and install new binaries,
 bin/apachectl stop
 #make install
 restart apache now
 bin/apachectl start
```

- 
- 
- If you have read my article - How to install php on windows then you must know by now that we have to change move the php.ini file to the proper location

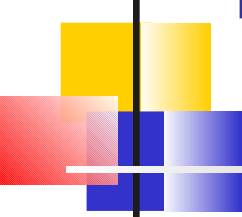
```
cd ../php
```

```
cp php5.ini-dist /www/conf/php5ini
```

and remove-dist from its name.

- Do you think we are done with PHP installation on Linux? well, fortunately yes.. we are ... now you can make a test page with

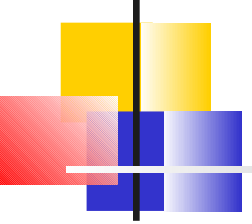
```
< ?php echo "PHP ! I'm gonna get you.."; ?>
```



# How to Install PHP on Windows

---

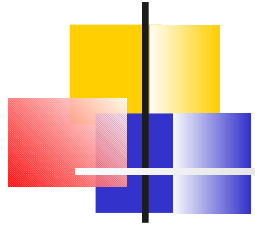
- Well, first of all visit [PHP Official website](#) and download Windows Binary Installation files. There are different versions of PHP the latest version is PHP 5.2.0 at the time of writing this article.
- One more thing, you will see two kinds of Windows Binary files .zip and .exe I am going to follow .zip file installation as .exe will not install PHP as Apache module on Windows. So if you are going to follow this tutorial then download .zip file.



## Contd

---

- Now create a directory for your PHP installation typically on “C:\PHP”, and create a temporary directory, may be “C:\PHP\temp” to unzip the files. As we don’t need all the files included in the .zip archive for PHP installation so first we are going to unzip the files in the temporary directory that we just created and then we will copy only those files to main “C:\PHP” directory which are needed for PHP installation.
- Now unzip the downloaded file to the temporary directory “C:\PHP\temp” and copy the following files to main PHP directory “C:\PHP”



## Contd

---

php.exe

php5ts.dll

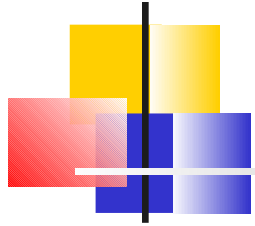
php5apache.dll

php5apache2.dll

- The file names will differ according to the version of PHP you downloaded. Remember we need php5apcahe.dll if you want to run PHP with Apache 1.3 version and php5apache2.dll for Apache 2.x version so you can skip one of them. You can find these files in the temporary directory (where you unzipped the zip archive) under 'sapi' folder.

# Contd

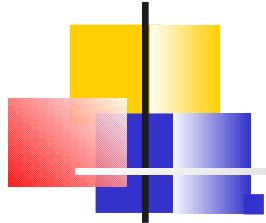
- Also if you want to use the different PHP extensions then copy files in extensions folder from temporary directory to main PHP directory though make sure your PHP is working fine before messing up with extensions' installation.
- Also MySQL is already enabled by now and you don't need to do anything extra to enable it with PHP.
- Now copy the php.ini-dist file to your Windows installation folder typically C:\WINDOWS and rename it to php.ini (remove -dist from the name)
- Congratulations! you are done with the PHP installation on Windows



# PHP Configuration

---

- If you want to use posted variables with their direct names e.g \$var1, \$name etc then you need to switch on the Global Variables in the php.ini file that we just moved to Windows installation folder. Open the php.ini file with notepad and search for this line  
register\_globals = Off  
change Off to On and you are done.
- Remember if you will not turn it on then you would have to access your posted variables with the REQUEST GLOBAL ARRAY reference i.e \$\_REQUEST['var1'] etc



# PHP Configuration Contd.

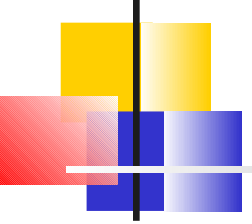
---

## Setting Up SMTP SERVER:

- If you want to send emails from your local PC using your ISP then you need to set up your SMTP server in the php.ini file , search for this code

```
[mail function]
;for win32 only
SMTP = localhost
;for wind32 only
sendmail_from= me@localhost.com
and change it accordingly to your SMTP server settings
e.g
```

```
[mail function]
;for win32 only
SMTP = mail.yourisp.com
```



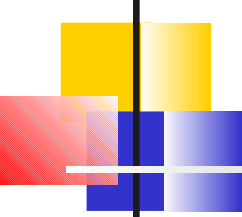
## - Contd

---

;for wind32 only

sendmail\_from= you@somemailservice.com

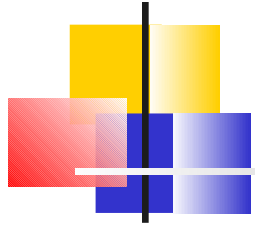
- If you are terribly confused with this SMTP thing then don't worry, You are a NEWBIE ! so just forget about it and once we are handling Mail in PHP
- then you will understand the need and use of SMTP server etc .



# HOW TO INSTALL APACHE ON WINDOWS

---

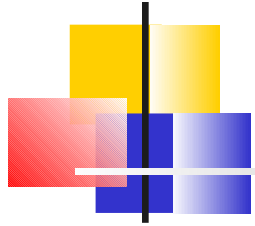
- We are proceeding well as I am getting your feedback on the previous articles of Fast PHP Tutorial Series and I am happy to hear that they are working and you are learning something...
- Till now we have learned “How to Install PHP On Windows” and “How to install PHP on Linux”. Lets start our today’s article which is about installing Apache on Windows so that it can support your PHP scripts on your PC.
- First of all download apache, Win32 Binary (MSI Installer), from <http://httpd.apache.org/download.cgi>



## Contd

---

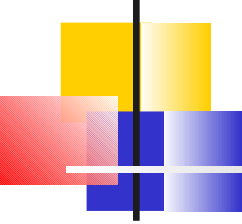
- When you will run the downloaded installation file make sure to put right settings info may be as follow,  
Network Domain: **localhost**  
Server Name: **localhost**  
Admin Email: **youremail@domain.com**
- You may get Anti-Virus or Windows XP SP2 security alert about running apache as service. Allow apache to run as service on your windows.
- You must run apache to execute PHP scripts and you can stop it optionally when you are not working with PHP and don't need apache to waste your system resources.



## Contd

---

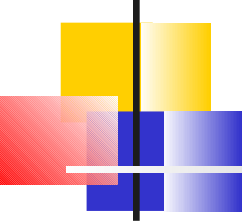
- After apache installation you will be able to see Apache Status Icon in your System Tray; sorry... what did you say? what is system tray? ... well, the area where your system displays clock (time) or display msn, yahoo and icq etc icons is called system tray.
- Apache shows a green kinda icon if it is running and red icon if stopped. you can take mouse on the icon to see the status on mouse-tip. You can also start and stop apache from Start Menu as Apache have made a menu from where you can also run Test Configuration to check your apache installation. If you click on Test Configuration and a dos window appears and then disappears it means installation is working otherwise it will display inform:



# Contd

---

- Now to test your Apache Installation on Windows open a browser window and type <http://127.0.0.1/> or <http://localhost/> in the address bar. If it shows you Apache home page it means your apache is working.
- Now you have to tell Apache where to pick your website pages, for this open the http.conf (apache configuration file) with notepad and search for DocumentRoot and put your website files exact location which may be,
- DocumentRoot “C:/PHP\_code/htdocs” now search for and paste your DocumentRoot path here too



## Contd

---

Remember whenever you make changes to Apache configuration file you need to restart apache to make the changes take effects. So, what are you waiting for... restart and get ready to dive deep into the PHP-Sea as your system is ready by now to start learning PHP.

## Easy Installers to Install PHP, MySQL, Apache on Win

- You have learned How to install PHP and Apache on Windows and Linux manually in my previous articles. Today I am going to tell you about ready made installers that make your lives easy to install PHP, MySQL and Apache on Windows by a single setup file.
- WAMP: Installs PHP , MySQL and Apache etc on Windows
- WAMP5 will allow you to discover all the PHP5 new features :  
Objet support, SQLite built-in, SimpleXML, exceptions.  
WAMP5 is build with standard conf files and most of the extensions. WAMP5 add-ons will add new features to your server like PHP4, Perl, Zend Optimizer...

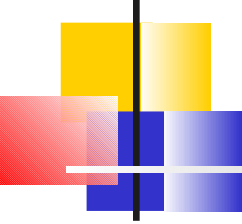
# Contd

- For further information please visit their official website at <http://www.wampserver.com/en/>

**the other well-known is**

- **XWAMP:** Installs PHP , MySQL and Apache etc on Windows

XAMPP is an easy to install Apache distribution containing MySQL, PHP and Perl. XAMPP is really very easy to install and to use - just download, extract and start.



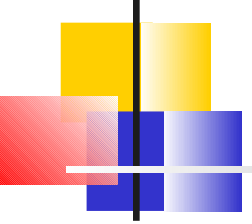
# Contd

---

- However, this is a dead open source project (at the time of writing this article) and is no more supported by the vendor so it contains old versions of Apache, PHP and MySQL.
- I hope by now it will be piece of cake for you to setup your PC to run PHP MySQL and Apache with Windows and Linux. Do it as soon as possible because in the coming articles we will start learning PHP.

# What are Tags?

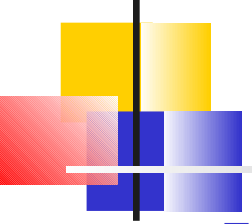
- Tags are the starting and ending point for a specific code snippet belonging to any scripting language. For example we close HTML codes between  
`<html>` and `</html>`
- similarly javascript code is enclosed like  
`<script language=javascript>`  
    `//all statements goes here`  
`</script>`
- ASP (Active Server Pages) uses  
`<%`  
    `'ASP code goes here`  
`%>`



# What are Tags? Contd

---

- And similar is the case for other languages.
- so you are thinking that why do we need tags? For this purpose first of all you have to understand the process of a web page display in your browser window i.e Internet Explorer (IE), Opera or FireFox (FF) etc etc



# Web page Parsing

---

- When you write any web page URL in your browser's address bar e.`http://www.fastcreators.com` and click enter; you in fact send a request to your web server that follows the URL and picks the contents from the target location of the requested web page.
- Remember, whatever you see in your browser is HTML output of the actual code of the requested web page and it might not be the actual code specially in cases when a web page uses a client-side technology or server side scripting to process information to show you the output of a page.



---

- **Code Explanation:**

The above PHP code is very simple and contains only three lines,

first it start with PHP starting tag and after the crap written after it, it finally close the code session with PHP ending tag. now lets discuss the crap

ECHO helps you to display any combination of string and variables on your page... wasn't it simple?

- **Syntax:**

echo "display message goes here";

everything that you want to display on the page must be enclosed between either double quotes or single quotes, php allows both.

# Contd

- The semi-colon is used to define the end of a statement in PHP. Your first quick verbal assignment.
- how will you output your “Hello YOUR NAME” message in php?

5..4...3...2...1

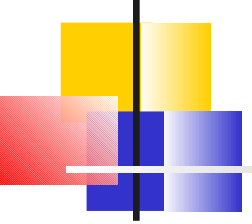
stop thinking and give me your answer... good work  
I know you have something inside you therefore you  
are learning PHP.

- yes, for my name it should be,

```
<?php
echo "Hello CSPL";
?>
```

# Contd

- To make sure things are working for you fine write different sentences and make a php page and browse it to check your php is working as well as you are understanding my lecture. Did you say how to browse a php page? well, make sure your PHP and Apache or other web server like IIS is running, then open a browser window and write [http://localhost/your-php-pages- folder/yourpagename.php](http://localhost/your-php-pages-folder/yourpagename.php) and this is how we execute PHP pages.
- Now close your eyes for five minutes and go through all major points that we have learned till now in this lecture...  
wake up guys ! ... lets talk about few new friends...

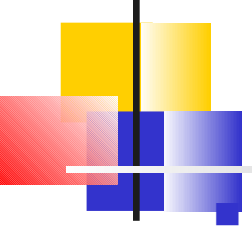


# Variables

---

- Variables are your friends, whenever you have something to store somewhere temporarily give it to the variables and they will keep it for you and whenever you need it call related variables and they will give it to you and you can change, add or delete the received information and return the new values to variables and they will not complaint at all ...
- Who shouted BE SERIOUS? Ok, so to define a variable, seriously, it gives you the facility to reserve some space in the memory for different values to use them later in your script's life time.

## Contd



For example, I want to save the message “Hello CSPL” in memory and then first display “PHP Tutorial” and afterward the message I stored; I will write the following simple programme,

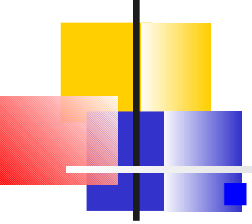
```
<?php
$message = "Hello CSPL";
echo "PHP Tutorial";
echo $message;
?>
```

- the output of this programme will be

PHP Tutorial Hello CSPL

# Contd

- No It is not a type but it is in fact how the output will look like, if you want the message to appear in the next line then put '\n' after first message i.e  
echo "PHP Tutorial \n";
- \n is used for Line Feed or next line. Remember on Linux the line feed is obtained by using \n however on windows you have to use \r\n
- \r stands for carriage return and these special characters are called Escape Characters, anyway don't confuse yourself we will study Escape Characters later in details.
- In PHP we define a variable with \$ (dollar) sign (dollars entered into our programming too )



## Contd

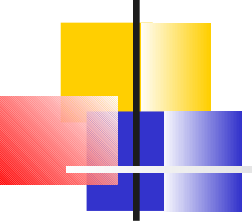
---

Let stick to our articles repository request now to understand the tags and parsing in depth.

**step 1:** <http://fastcreators.com/article/index.php> is requested

**step 2:** Server that hosts this web page is requested to send the contents attached with this page.

**step 3:** web server examine the code for any additional processing, as our articles repository's back-end is written in PHP therefore the web server will find the PHP TAGS in different places and as soon as it finds the PHP starting tag it will start sending those statements to the PHP parser Engine unless it finds the PHP ending tags for that specific code set.

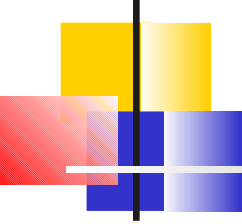


# Contd

---

## step 4:

Web server asks the PHP parser engine to jump in and generate the HTML output from the php code in the page. As all the articles are stored in a MySQL database so the index.php code has necessary statements to fetch the latest articles info from the database. Hence all the information is fetched and the desired HTML output from all this PHP-MySQL integration and processing is returned to the web server from PHP Parser Engine and web server finally show the page that you see by visit my Articles Repository at So this is how important are the tags for any language in fact they informs the web server that they need to be processed by some parser engine before the final output is sent to the browser.



# Contd

---

- PHP Starting and Ending Tags:

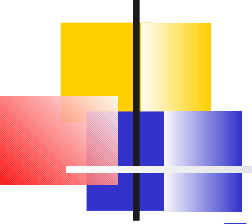
Similar to other languages PHP code starts with `<?php` tag and ends on `?>`

Lets write our very first PHP code:

```
<?php
```

```
 echo "PHP ! I am gonna get you.";
```

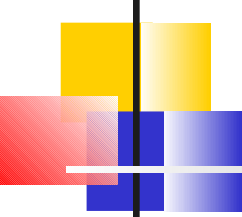
```
?>
```



# Web page parsing variable contd

---

- PHP is a loose-typing scripting language and it doesn't have any hard and fast rules for variables like C or Java languages have. You can define a variable with \$ sign and parser itself determines the data-type of this variable at run time.
- However PHP has seven types of variables, and all but one hold a specific class of information. The seven types are: strings, integers, floats, booleans, arrays, objects, and resources.
- Strings hold characters (literally: a string of characters) such as "a", "abc", "Jack and Jill went up the hill to fetch a pail of water", etc. Strings can be as short or as long as you want - there's no limit to size.



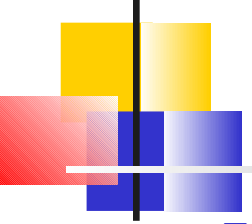
# Web page parsing contd

---

- Integers hold whole numbers, either positive or negative, such as 1, -20, 55028932, etc. There is a maximum limit to the size of integers - any numbers lower than -2147483647 and any numbers higher than 2147483647 are automatically converted to floats, which can hold a much larger range of values.
- Floats hold fractional numbers as well as very high integer numbers, such as 4.2, 1.000000001, and 2147483647000.
- Booleans simply hold true or false. Booleans are, in fact, just integers behind the scenes - PHP considers the number 0 to be false, and everything else to be true.

# Web page parsing variable contd

- Arrays are a special variable type in that they hold multiple values. Arrays can be quite complicated, and so are covered in detail in their own chapter.
- Objects are complex variables that have multiple values, but also their own functions. Objects are also very complicated, and, like arrays, are covered in their own chapter.
- Resources are anything that is not PHP data - this might be picture data you have loaded from a file, the result of an SQL query, etc. Resources are used like any other variable, with the key difference that you should generally remember to free up your resources when you are finished with them.



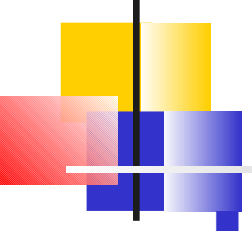
# Operators and operands

---

- See the following PHP Code,

```
<?php
 $salary = 3000;
 $bonus = 1000;
 $total_salary = $salary + $bonus; // this line will be
 explained
 echo "Total Salary is: " . $total_salary; // output:
 Total Salary is: 4000
?>
```

- One other rule you should remember is that you cannot use spaces, dot or dashes in your variables name however you can use an under score.



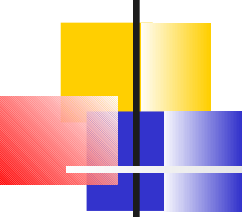
# Operators and operand contd

---

Operands are the entities that have some values in them or you can say the variables and operators are the symbols that have special meanings in the language for example addition operator ( + )

- Also in echo statement I used something new; Dot Operator: it is used for concatenation or to join different entities to create a final string for echo to print on the page.
- The following will display “Hi Haroon, How are you?”

```
<?php
 $question = "How are you?";
 $name = "CSPL";
```



# Operators and operands contd

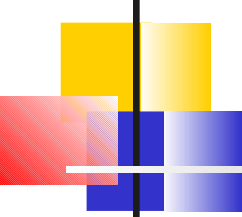
---

- **concatenation:**

```
echo "Hi" . $name . ", " . $question;
?>
```

It helps you to glue different values into a single statement

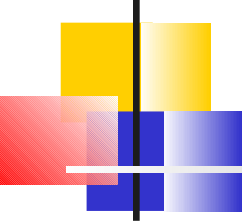
- With this we will end our today's lecture. Please practice ! I agree with Shakespear, "Practice makes a man perfect" and programming is all about practice. To learn syntax of a programming is not even the start of the story you can call yourself a programmer only when you are able to create logic to solve some computer problem with less code and fast execution time and you can achieve this through practice. Programming practice polish your skills and gives you new ideas to solve different problems through logic.



# OPERATORS IN PHP

---

- Today we are going to discuss different types of operators used in PHP. I hope you remember the basic definition of the operators and operands from my last article ( [PHP Programming Basics](#) ) and I am not going to explain it again as we had promised in the very first article of this [Series of PHP Tutorials](#) that we will not see behind and this is just to compel you to concentrate on each and every article of this series and practice .
- PHP offers the following operators to perform different operations on the operands.



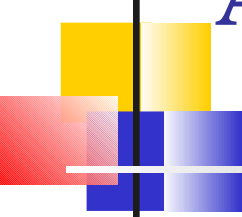
# 1. ARITHMETIC OPERATORS

---

- Arithmetic operators enable you to perform different mathematical operations on different values.
- Below are the quick details of arithmetic operators,  
**Addition:** it is denoted by plus sign ( + ) and is used to add two values e.g  $2 + 3 = 5$   
you can add variables as following,

```
<?php
 // addition
 $var1 = 4;
 $var2 = 5;
 $total = $var1 + var2;
 echo "Total = " . $total;

?>
```



# ARITHMETIC OPTRS CONTD

---

- **Subtraction:**

it is denoted by minus sign ( - ) and is used to subtract first value from the second e.g  $2 - 1 = 1$

you can subtract two variables as following,

```
<?php
// subtraction
 $var1 = 8;
 $var2 = 5;
 $total = $var1 - var2;
 echo "Total = " . $total;

// Total = 3
?>
```

# ARITHMETIC OPTRS CONTD

## multiplication:

it is denoted by Asterisk sign ( \* ) and is used to multiply two value e.g  $2 * 3 = 6$

you can multiply two variables as following,

```
<?php
 // Multiplication
 $var1 = 8;
 $var2 = 5;
 $total = $var1 * var2;
 echo "Total = " . $total;
// Total = 40
?>
```

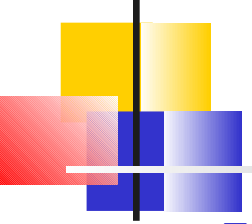
# ARITHMETIC OPTRS CONTD

## Division:

It is denoted by forward slash sign ( / ) and is used to divide first value on the second e.g  $2 / 2 = 1$

you can divide two variables as following,

```
<?php
// Division
$var1 = 15;
$var2 = 5;
$total = $var1 / var2;
echo "Total = " . $total;
// Total = 3
?>
```



# ARITHMETIC OPTRS CONTD

---

## Modulus(Division Remainder):

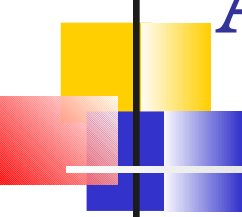
It is denoted by percentage sign ( % ) and is used to give the division remainder e.g  $3 \% 2 = 1$

you can find the modulus of two variables as following,  
<?php

```
// Modulus
```

```
$var1 = 15;
$var2 = 5;
$total = $var1 % var2;
echo "Total = " . $total;
```

```
// Total = 0
?>
```



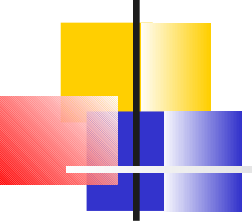
# ARITHMETIC OPTRS CONTD

---

- **Increment:**

it is denoted by double-plus sign ( ++ ) and is used to increase value of one variable by one e.g

```
<?php
// Increment
$counter = 15;
$counter ++; // 16
$counter ++; // 17
echo "Counter = " . $counter;
// Counter = 17
?>
```

- 
- 
- Remember there are two types of incrementation,
  - a) **Prefix Increment**: in which the increment operator comes before the variable name i.e ++\$var;

In this type of increment the variable's value is incremented and is available for the immediate use e.g

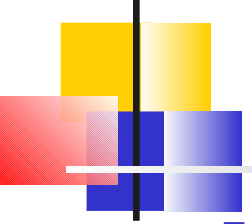
```
<?php
// Prefix Increment
$counter = 15;
echo "Counter = " . ++$counter;
// Counter = 16
?>
```

# Types of Increment optrs

## contd

- b) **Postfix Increment** in which increment operation is performed however the resultant value is not available for the immediate use and can be used only when the control access the variable in the next run e.g

```
<?php
// Postfix Increment
$counter = 15;
echo "Counter = " . $counter++;
// Counter = 15
?>
```



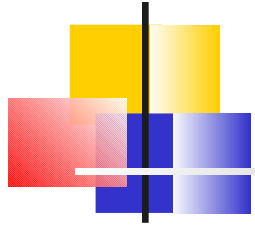
# Operators in php contd

---

- **Decrement Operator:** it is denoted by double-minus sign ( — ). It is very similar to the increment operator however it decrease the value by one.
- **Assignment Operators:** Assignment operators are used to assign new values to a variable. We have the following assignment operator available in PHP,

<?php

```
$var1 = 3;
$var2 = 5;
$var1 = $var2; //var1=5
$var1 += $var2; //var1=8
$var1 -= $var2; //var1= -2
$var1 *= $var2; //var1=15
$var1 /= $var2; //var1=0.6
$var1 %= $var2; //var1=3 ?>
```



## ■ Comparison Operators:

Comparison operators are used to compare two values. We can use following comparison operators in PHP,

( == ) is equal to

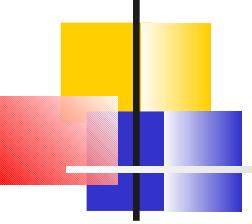
( != ) not equal to

( > ) greater than

( < ) less than

( >= ) greater than or equal to

( <= ) less than or equal to



# Logical Operators

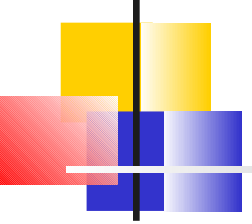
---

- **Logical Operators:**

- **AND ( & & ) Operators:** you can use both 'AND' and double-ampersand sign ( && ) for this operator in PHP. The basic logical use of this operator in a rough pseudocode could be

***If first condition is true & & second condition is also true***

- **OR ( || ) Operators:** you can use both 'OR' and double-pipe sign ( || ) for this operator in PHP. The basic logical use of this operator in a rough pseudocode could be



# Contd

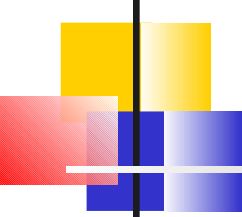
---

***Either the first condition is true || the second condition is true***

- NOT ( ! ) Operators: you can use both 'NOT' and Exclamation Mark (Mathematician will call it Factorial Sign) sign ( ! ) for this operator in PHP. The basic logical use of this operator in a rough psuedo code could be

***First value is NOT equal to the second value***

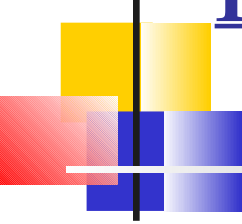
- Again do not confuse the use of these operators but just learn them by heart and I will use these all operators in my up coming articles for more complex PHP programmes and then your all doubts will be removed hopefully.



# Contd

---

- This is one of the most important articles of this PHP Tutorials Series. Please concentrate hard on this articles and try to understand all issue discussed today and feel free to ask if you have any question and remember if you cannot understand this foundation lectures of PHP it will be very hard for you to survive when we will ‘BURN IT’ as I had promised with you that I will take you the Extreme PHP Programming and the Only rule of our game is “Don’t look behind” so you will not be able to go back to these articles again so learn these things now this article will make your strong foundation towards complex programming.

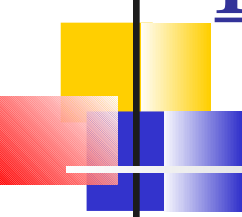


# Decision Making Using IF-ELSE In PHP

---

## ■ Decision Making Using IF-ELSE In PHP

- Today we are going to study the tools that will help you to write Decision Making Codes in php. For example, when you sign in to orkut.com you see a message 'hello username' on the upper right of every page. Similarly when you login to gmail if you provide wrong username or password you see an error message and if you submit accurate information you are taken to the inbox.
- If you think deep on these events, this is what we call 'decision making code'. you write intelligent code that can make decision on user's input and can act accordingly to the input to produce required output.



# Decision Making Using IF-ELSE In PHP

---

- PHP provides us IF - ELSE statements to write decision-making-code. Suppose I want to make a page that asks for the user's name and password and if the username and password are wrong it displays welcome message otherwise unauthorization message.
- To read the username and password from xml file (wow! I like that!)
- To read the username and password from text file (faster than database)
- To read the username and password from MySQL database (or Oracle, PGSQL, MySQLi, Access)

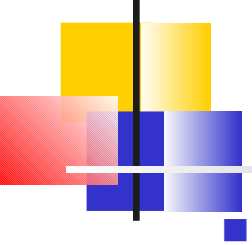
# IF-ELSE SYNTAX

- **IF-ELSE Syntax:**

The basic IF-ELSE syntax is as follows,  
if(condition)

```
{
 //block of code
}
else
{
 //block of code
}
```

- If the condition that we knit together by the help of logical and comparison operators if found true then the code between IF braces will be executed otherwise ELSE code block will be taken on.



# TRIM FUNCTION

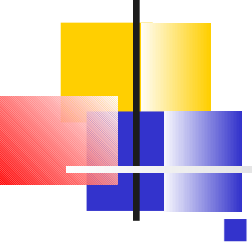
---

Trim function is used to remove blank spaces from start and end of the variable values to ensure user is not filling blank form with spaces or unintentionally may have pressed blank spaces. It is handy to apply this function while receiving form values.

## Fetching Values From Global Arrays:

- There are two ways to fetch variables from sending page
  - 1) directly be referring to the variable name using global arrays (`$_POST`, `$_GET`)

# TRIM FUNCTION CONTD

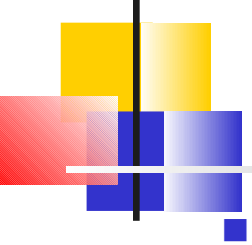
- 
- It depends on your php.ini settings if the GLOBAL settings are ON then you can refer to the variable name directly otherwise you have to use Global arrays.
  - I'd recommend the use of global arrays because mostly server's configure php to switch of the global variables for security reasons and it is better to fetch values using global arrays so that your code can run on different servers with different php configurations without any crash.

# Multiple if-else command

- You can use more than one set of if else as follow,

```
if(condition)
 //code here
elseif (condition)
 code here
elseif(condition)
 //code here
else
 code here
```

- you can write multiple sets of IF-ELSE

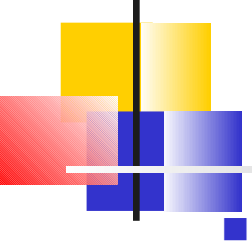


# Multiple if-else command contd

---

- For example if you are asked to make a resultsheet for the students and to calculate the grade depending on the percentage so you will need more than one IFs. ELSEIF is used to attach a second IF condition with the previous IF so that if the previous IF is false the code will check the next IF. Remember if the first IF statement is true then control will not come to the remaining IFs or Else. Last else is executed if all IF and elseif conditions are false.
- Also you have may noticed that I didn't use opening and closing braces for IF and ELSE it is because when you have single line after IF or ELSE then it is not necessary to put opening and closing braces however it is a good practice to put them.

# ASSIGNMENT



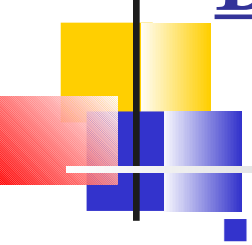
---

**Assignment:** Make a result sheet system based on two pages,

- a) **score page:** in this there should be five fields where user can submit number in five different subjects
- b) **result page:** this page will receive the form and will display total numbers, grade and percentage based on the submitted score.

You can email me your assignments or post in comments to this thread.

In the next articles I will show you how to use SWITCH statement for Decision Making Code.



# Decision Making Using SWITCH In PHP

---

- Today I am going to discuss Switch Statement. The basic concept of Switch is similar to the IF-ELSE statement; it helps you to write decision making code just like IF-ELSE but it is a little more organized, elegant and handy when dealing with complex logics as compare to IF-ELSE.

Remember, you can convert every IF-ELSE programme to Switch syntax but vice versa is not possible in some situations.

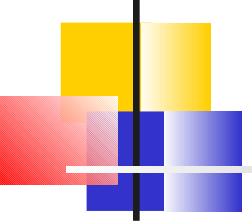
- Remember one more thing, it is not necessary to convert IF-ELSE to Switch and vice versa, just use whatever suits to your needs, though when handling the complex and large decision making code I would suggest to use Switch.

# SWITCH SYNTAX

- Let's have a look at its syntax.

- Switch Statement Syntax:

```
switch (input)
{
 case condition1:
 code to be executed if input = condition1;
 break;
 case condition2:
 code to be executed if input = condition2;
 break;
 default:
 code to be executed
 if input is different
 from both condition1 and condition2;
}
```



# BREAK STATEMENT

---

- **What is break statement:**

Break statement breaks the flow of a certain code block in which it resides and take the control out of the block. You can use break without switch statement too for example to take the control out from a FOR loop (will discuss later) or IF statement.

# Example of Switch statement

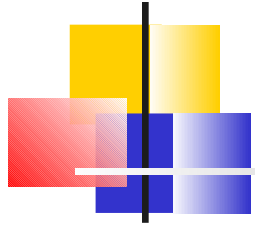
Let's write a simple programme to understand the use of Switch statement in PHP.

```
<?php
 $site = "Fast Creators";
 switch ($site)
 {

 case "Fast Creators":
 echo "www.fastcreators.com";
 break;
 case "How To Forge":
 echo "www.howtoforge.com";
 break;
 case "PHP Official Site":
 echo "www.php.net";
 break;

 default:
 echo "Input did not match with any case";

 }
```



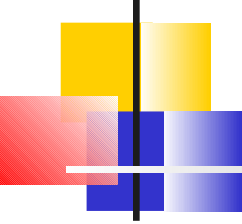
- **Assignment:**

Make a form that has different countries names in a drop-down list. After you select a country and submit the form the accepting page should use Switch statement to check the country name and then display its Capital, Currency Name and National Language.

# LOOPS IN PHP

- There are certain conditions in which you need to execute the same block of code again and again. For example if you want to print ten consecutive equal signs in three lines to make a separator then you could do it with different methods. One could be to use three echo statements and put ten equal signs in each i.e

```
<?php
 echo "============" . "
";
 echo "============" . "
";
 echo "============" . "
";
?>
```



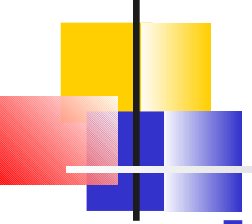
# LOOPS Contd

---

- Apparently it's not that hard to do so with echo statement but imagine if you have to print 100 such lines that means you will write  $3 \times 100 = 300$  echo statements. Loads of typing and hassles to organize them. Here we can use Loops to write few lines which will print more output depending on our loops condition.
- Before we get to the actual loops explanation lets understand the basic working of loops.

# BASIC CONCEPT OF LOOPS

- We use braces `{ }` to make block of code that we want to execute for a specific number of times. This is called the body of loop. Every loop depends on a condition. It keeps executing the block of code unless the condition is met true or stand false (depends on the loop's type that we will discuss in a moment).
- There is a terminating point for each loop that ends the loop when the condition is satisfied. Like in our earlier discussed example if we want to display hundred lines of ten equal signs each then we have the following attributes that will make up our loop in theory;



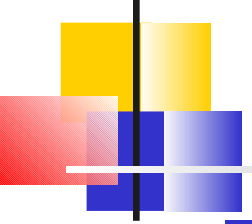
# Basic concept of loop contd

---

- 1) **Condition:** hundred lines: keep printing unless hundred lines are printed.
- 2) **Body of the loop:** in which we will put the printing stuff that is echo statement to print equal signs plus some additional contents that will help our loop to execute for the desired number of timing (will discuss in a moment)
- 3) **Terminating Point:** when hundred lines are printed the loop must terminate
- 4) **Loop Run:**  
Every time the block of code (body of loop) executes the loop completes its one RUN.

# Contd

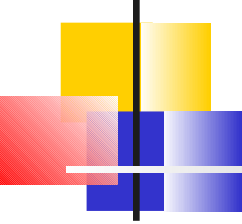
- Isn't it easy? Yeah it is and it will be more easy for you once we discuss the actual loops. So let's do it!
- Remember, I am using these terms from my own dictionary because I will use them to explain things otherwise these terms or not standard and attached with loops definitions you can use whatever theory you want to explain these terms in your own words.



# TYPES OF LOOPS

---

- Types Of Loops:  
PHP offeres four types of loops
  - FOR Loop
  - While Loop
  - Do-While Loop
  - Foreach Loop
- Now we will discuss them one by one.



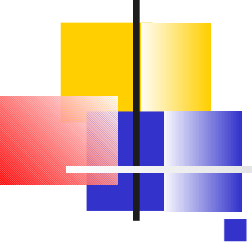
# For loop

---

- Basic Syntax of For Loop is as follows,  
FOR Syntax:

```
for (initialization; condition; increment /
 decrement)
{
 //body of loop
 // increment / decrement
}
```

# For loop contd

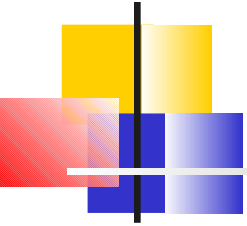


Here '**for**' is the keyword following by parents that contains three attributes

**Initialization:** The initial value of the loop from where the loop will pick the starting value

**Condition:** The condition that the loop will check after each loop run and will decide whether it has to execute another loop run or terminate

**Increment / Decrement:** This portion will either increment or decrement the initial value to continue

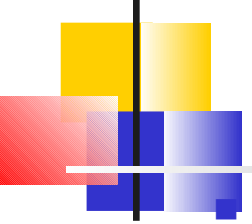


- Let's write the first programme that will use FOR loop to print equal sign (=) ten times in each row for hundred lines

```
<?php
```

```
for($i=1; $i<=100; $i++) {
 echo "===========" . "
";
}
```

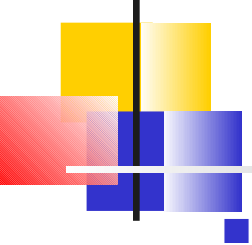
```
?>
```



---

## Now let's analyze the above code.

- First the control picks the loop initial value as 1 and compare it with the condition which says that \$i value can be less or equal to 100 which stands true because \$i current value is 1 so the loop completes its first run in which it displays one line of equal signs and increment \$i value by 1. On reaching the closing braces the control again goes back to the Condition and compare \$i value which is 2 now. The condition again stands true as 2 is less than 100 and loop completes another run. Similarly it performs the same operation unless the \$i value comes 100.
- When \$i is 100 then the condition stands true and loop completes its hundredth run and increment \$i to 101. Now on comparison the condition stands false because 101 is not equal to 100 nor less than it so the loop reaches termination point and the control is sent out of loop body.



---

**Loop Variable:** The variable involved in loop initialization, condition and increment is known as Loop Variable. So what's our loop variable in the above example? yeah, it is \$i.

Let's perform the same operation with FOR loop but with a different method.

```
<?php
```

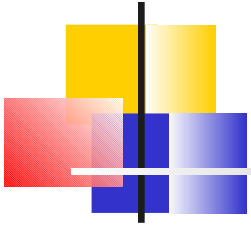
```
 for($i=100; $i>=1; $i--)
```

```
{
```

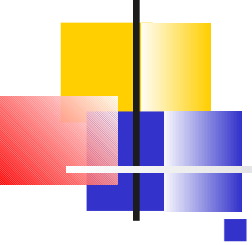
```
 echo "===== " . "
";
```

```
}
```

```
?>
```



- Now our initial value of loop variable is 100. So you will be thinking that how we are going to print the line for hundred times? We will do so because we also changed the Condition that is now comparing \$i unless it is greater or equal to 1. And the third attribute is now decrement and not increment so the loop variable will start from 100 and will decrease its value by 1 for each loop run i.e 99,98,97 ..... , 1 and when it will reach 0 the condition will stand false and the loop will terminate.

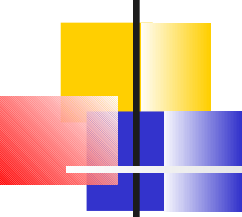


---

Anyway, you should remember one more thing that it is not necessary to increase or decrease loop variable by 1 each time. You can use any increment or decrement value you like. For example I can display 50 lines from the same above code just by decreasing the loop variable by 2 in each run i.e

```
<?php
```

```
 for($i=100; $i>=1; $i-=2)
 {
 echo "===== " . "
";
 }
?>
```

- 
- 
- You can also decrease the value by 2 like `$i=$i-2`. Here you must notice yet another thing that increment or decrement part of the loop doesn't use semi-colon. There could be different styles to write the FOR loop code e.g

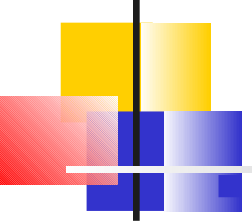
```
<?php
```

```
 for($i=100; $i>=1;)
 {
 echo "===== " . "
";
 $i=$i-2;
 }
?>
```

# WHILE LOOP

- FOR loop is a definite loop because you know the number of loop runs before the loop starts. However WHILE loop is indefinite loop because it works on mainly a condition that can stand true for unknown number of loop runs.
- Let's explain it with a broader example. Suppose you have 1000 students names and you want the loop to read each name and perform some action unless the name appears as ROB.

# While loop contd



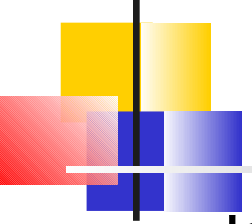
---

```
while(name != ROB)
{
 // While body
}
```

that means unless name variable is matched to be ROB the loop should carry on.

- Now let's stick with our equal signs example to perform the same operation with WHILE LOOP.

```
<?php
 $i=1;
 while($i <= 200) {
 echo "===== \n";
 $i=$i+2;
 }
?>
```



# While loop contd

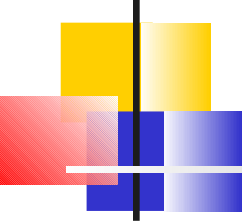
---

Indefinite loops are also called iterative loops because real time indexing is not constant. For a very good understanding of definite and indefinite / iterative loops you can study

<http://www.cs.duke.edu/~ola/patterns/plopd/loops.html#defir>

What if I write something like

```
<?php
 $i=2;
 while($i<1)
 {
 echo "how many loop runs this will have?";
 }
?>
```



# Do While Loop

---

- Do-While loop will run the loop atleast once before it checks the loop condition and hence it's handy in the situation where we need to run the loop for the first time without checking the condition.

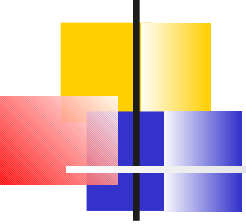
the basic structure of do-while loops is as follow,

```
do
{
```

```
 //block of code that will be executed in each loop
 run
```

```
}
```

```
while(condition);
```



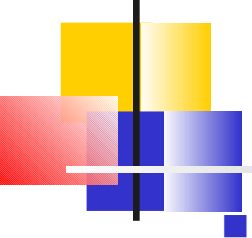
# Do While Loop contd

---

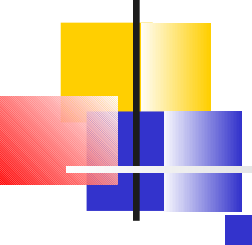
- How many times my name will be printed from the following code?

```
<?php
 $i=2;
do
{
 $i++;
 echo "Haroon";
}
 while($i<1);
?>
```

# Do While Loop Contd

- 
- The first time control reaches the while condition the \$i value is 3 and it stands false and the loop is ended however you already printed my name once because do-block of code executed already before reaching the condition.
  - do-block of code is made using keyword 'do' and pair of braces.
  - So by now you have learnt three loops types in php out of four.

The basic concept of the functionality of all loops is same that contains initialization, condition check and increment / decrement, termination of loop, body of loop etc etc. Though the syntax and process flow are different.

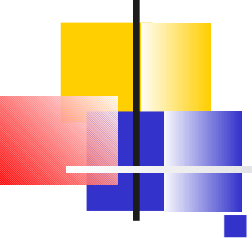


# ASSIGNMENT

---

## Assignment:

Make a form that has one field where user can submit a digit and on the receiving page display its tabl



# Arrays in PHP

---

- Today we are going to discuss **ARRAYS**. We will learn its syntax, its different types, the different built-in array functions that help to perform different tasks related to arrays quickly and different practical examples explaining the use of arrays in PHP.
- By the end of this article you should be able to:
  - Define Arrays
  - Use different types of Arrays
  - Use different Built-in functions related to Arrays  
e.g list(), each(), var\_dump, print\_r,  
var\_export()
- Write basic as well as complex php code using Arrays.
  - Working with multidimensional arrays
  - Understanding Foreach loop with the help of different examples

# Arrays in PHP Contd



- **Proper Array Definition:**

“Collection of different variables under the same label to keep values organized and easily accessible for processing”.

- **Array Syntax:**

`$arrayName = array ("value1","value2","value3");`

We use \$ sign to give an array name like we do for variables, after that an equal sign and then keyword ‘array’ that tells the parser that we are working with arrays and then different values within parenthesis and each value enclosed in double quotes separated by comma.

# Example

- PHP Example to define and print arrays:

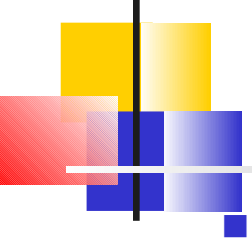
```
<?php
$Top3Sites = array
("fastcreators.com","howtoforge.com","scriptsbible.com
");
print_r($Top3Sites);
?>
```

The output of the above programme will be.

Array

```
(
 [0] => fastcreators.com
 [1] => howtoforge.com
 [2] => scriptsbible.com
)
```

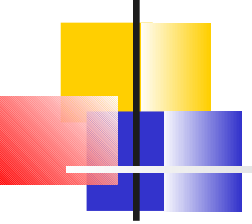
# Arrays in PHP Contd



Please note that we use `print_r` to print an array because you cannot print an array with `echo` or `print` function (both are used to display output) though you can use `echo` or `print` to display single items from the array e.g.:

```
echo $Top3Sites[0]; //fastcreators.com
```

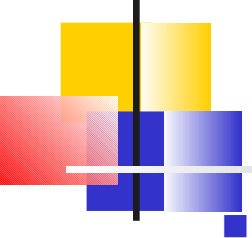
- Remember that the index number starts from 0 and not 1.
- Each value of the array get a unique ID which is known as INDEX NUMBER.
- The other way could be to define three different variables and then assign them values and use different statements to display them. It could be acceptable in case of three values for the sake of argument though will be rejected when we are dealing with fifty or hundred values.



# TYPES OF ARRAYS

---

- There are three different types of arrays in PHP:
  - a) **Numeric Array**: An array with a numeric ID key.
  - b) **Associative Array**: An array where each ID key is associated with a value.
  - c) **Multidimensional Array**: An array containing one or more arrays.



# Numeric Array

---

**Numeric Arrays:** Numeric arrays use integer / numbers as their index number to identify each item of the array. The example we discussed above are numeric arrays as they have integer values as index numbers for each item.

```
<?php
```

```
$colours = array("white","black","blue");
```

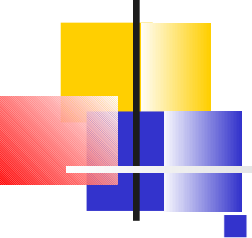
```
print_r($colours);
```

# Numeric array contd

■ output will be

```
Array
(
 [0] => white
 [1] => black
 [2] => blue
)
*/
?>
```

- In the above output you can see the index numbers for white, black and blue are 0,1,2 respectively which are numeric values and hence we call such arrays numeric arrays.



# Associative Array

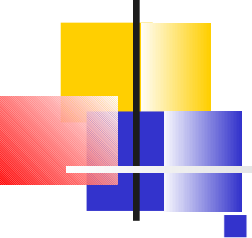
---

Sometimes it's better to use the index name instead of index number for example if you want to save three students' names and numbers so your best option will be to use each student's name as index value for the array and his numbers as the values, behold on the example below,

```
<?php
```

```
$students['Ganesh'] = 90;
$students['Hanuman'] = 60;
$students['Paras'] = 40;
```

```
?>
```



# Associative Array contd

---

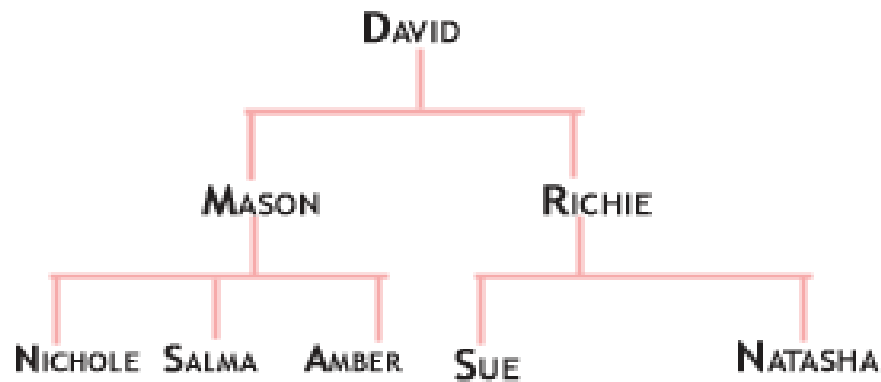
- When you submit a form using POST or GET method you get a similar associative array on the receiving page that contains the name of each form field as array index and its value as index value. Try to make a HTML form with some fields and post it and on the receiving page print the global arrays like

```
print_r($_POST);
print_r($_GET);
```

- and you will see the associative array.

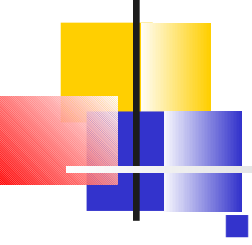
Associative Arrays are more easy to handle and to process information especially dealing with complex form submission and dynamic values from database etc.

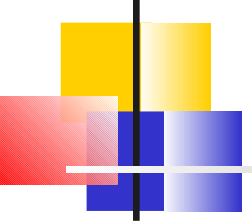
# Multidimensional Arrays



David's Family Tree Vs Multidimensional Arrays

# Multidimensional Array contd

- 
- A multidimensional array can contain arrays within itself and the sub arrays contain more arrays within them.
  - Let's move to a real world example to understand the concept of multidimensional arrays:
  - David has two sons Richie and Mason. Richie has two daughters Sue and Natasha while Mason has three daughters Nichole, Salma and Amber.
  - Now they family tree is as follows:

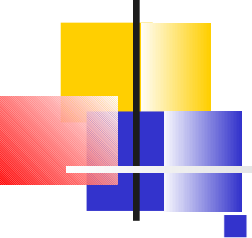


# Multidimensional array contd

---

- If we want to display David's family tree with a multidimensional array in PHP then we can define an array as below:

```
$david = array
(
 "richie"=>array
 (
 "Sue",
 "Natasha"
),
 "Mason"=>array
 (
 "Nichole",
 "Salma",
 "Amber"
)
);
```



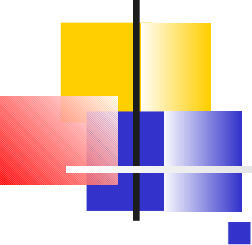
# FOR EACH LOOP IN ARRAY

---

■ Foreach loop is used for arrays to take each next value from array and perform whatever action is required on it.

- The basic syntax of a for each loop is as below:  
foreach ( \$arrayname as \$temporaryVarName )

```
{
 // foreach loop body
 //$temporaryVarName holds the array values and
 //on each loop-run it will pick the next available
 value from the array
}
```



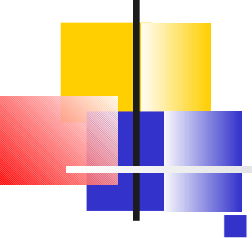
# For Each loop contd

---

Let's write a programme using a foreach loop to understand its working in depth:

```
<?php
```

```
$students = array(); // define array students
//assigning values
$students[0] = "Ganesh";
$students[1] = "Hanuman";
$students[2] = "Paras";
$students[3] = "Ajit";
//echo the array values in one go
foreach ($students as $std_name)
{
 echo $std_name . "\n";
}
```



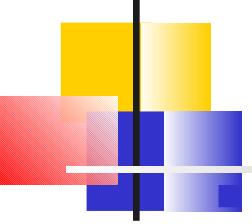
---

In the above code the `$std` name works as a temporary variable to get each value of array. On every loop-run the next available value of the array overwrite the existing value of the `$std` name and then `$std` name points to the current fetched value. So it's just like walking through your array values one by one.

The output of the above code will be:

Ganesh  
Hanuman  
Paras  
Ajit

As we are using `\n` (line feed) to display each name on the next line.

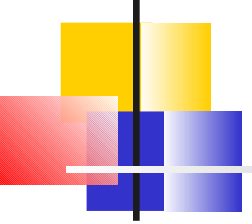


---

The foreach loop terminates when there is no value available next that is it reaches the last value of the array which is NAYYAR in our case.

- You can break the loop anytime using break statement if required (we have discussed break statement in the DECISION MAKING USING SWITCH IN PHP).
- For example if you want to break the loop if the name is julie then you can include an IF statement in the above code as below:

```
<?php
 foreach ($students as $std_name)
 {
 if($std_name == "julie") break;
 echo $std_name . "\n";
 }
```



# Important Arrays Function

---

- **1) Save print\_r function's output**

We have previously used print\_r to display the array values, though we can add an extra argument to print\_r and save its output to a variable e.g.:

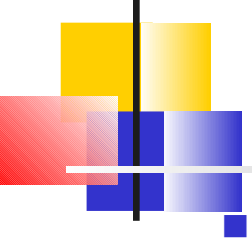
```
<?php
```

```
$myarray = array("Apples", "Oranges", "Pears");
$output = print_r($myarray, true);
print $output;
```

```
?>
```

The print\_r will display the array itself if you will not use

TRUE - the second argument however it will store its output in the variable if the TRUE is provided.



## 2) How to find the size of an Array?

Size of an array means how many values an array contains... if you want to find the number of values of an array you can use the COUNT function as follows:

```
<?php
```

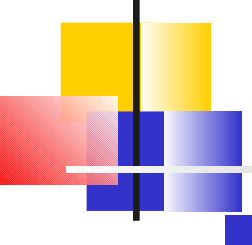
```
$myarray = array("Apples", "Oranges", "Pears");
```

```
$size = count($myarray);
```

```
print $size; // 3
```

```
?>
```

The above programme will display 3 because we have three elements in the array.

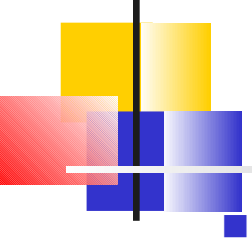


### 3) var\_dump Function:

---

There is a similar function to `print_r()`, which is `var_dump()`. It does largely the same thing, but

- a) prints out sizes of variables,
- b) does not print out non-public data in objects, and
- c) does not have the option to pass a second parameter to return its output.

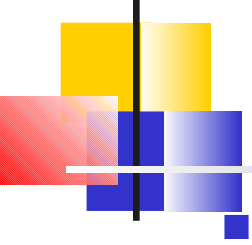


---

For example, altering the first script to use `var_dump()` rather than `print_r()` would give the following output:

```
array(3)
{
 [0]=>
 string(6) "Apples"
 [1]=>
 string(7) "Oranges"
 [2]=>
 string(5) "Pears"
}
```

- In there you can see `var_dump()` has told us that the array has three values, and also prints out the lengths of each of the strings. For teaching purposes, `var_dump()` is better as it shows the variable sizes, however you will probably want to use `print_r()` in your own work.



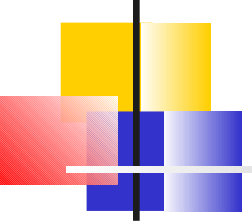
#### 4) **var\_export** Function:

Also, there is the function `var_export()`, which is similar

to both `var_dump()` and `print_r()`. The key difference with `var_export()`, however, is that it prints out variable information in a style that can be used as PHP code.

For example, if we had use `var_export()` instead of `print_r()` in the test script, it would have output the following:

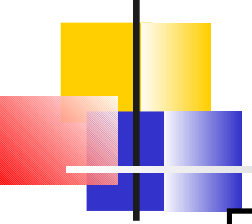
```
array
(
 0 => 'Apples',
 1 => 'Oranges',
 2 => 'Pears',
)
```

- 
- 
- Note there is an extra comma after the last element, however this is ignored by PHP and you can copy and paste that information directly into your own scripts, like this:

```
<?php
```

```
 $foo = array
(
 0 => 'Apples',
 1 => 'Oranges',
 2 => 'Pears',
);

?>
```

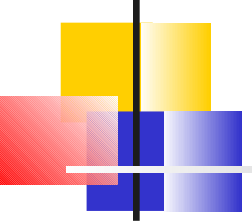


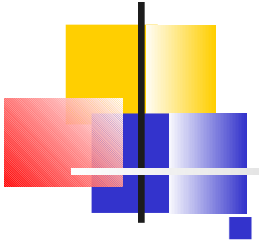
---

## 5) array\_shift Function:

- The array\_shift function removes the first element of an array from the array and stores it in the receiving variable.
- For example if I want to remove Apples from the array used in previous programmes and store it in a different variable then I can write a programme as below:

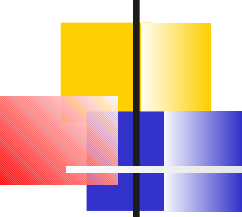
```
<?php
 $myarray = array("Apples", "Oranges",
 "Pears");
 $fruit = array_shift($myarray);
 echo $fruit; // Apples
 print_r($myarray); // Oranges and Pears will
 be displayed without Apples
?>
```

- 
- 
- It is a very handy function if you have a multidimensional array posted from a form and you want to fetch the sub-array from the main array. But remember when you fetch out a sub-array from the main array using `array_shift` then the receiving variables (`$fruit` in previous) will not be a normal variable but an array.
  - I will also suggest you to study three other useful functions related to `array_shift` on PHP Official Manual at
    - `array_unshift`
    - `array_pop`
    - `array_push`



- PHP offers a very long list of different useful array functions that you can find at [PHP OFFICIAL MANUAL](#).
- One thing I want to mention here is that the foreach loop is not the only way to process arrays - you can do it using other loops e.g FOR LOOP or list() and each() functions.
- To process an array through you will need to find the total number of the values in the array, i.e the size of array as below:

```
<?php
 for ($i = 0; $i < count($array); ++$i) {
 print $array[$i];
 }
?>
```

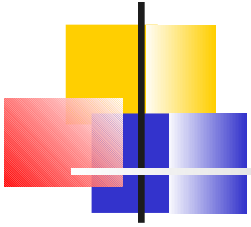


# Process Arrays WITH List () Function

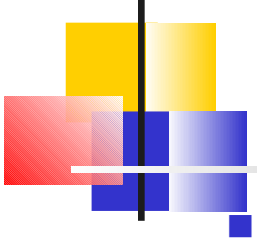
---

- PHP offers the LIST() function that can be used to go through array values easily and quickly as below:

```
<?php
 while (list($var, $val) = each($array))
 {
 print "$var is $val\n";
 }
?>
```



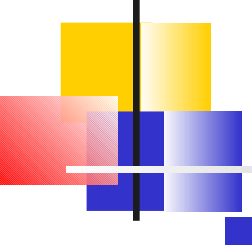
- List() is a function that does the opposite of array() - it takes an array, and converts it into individual variables. Each() takes an array as its parameter, and returns the current key and value in that array before advancing the array cursor. “Array cursor” is the technical term for the element of an array that is currently being read. All arrays have a cursor, and you can freely move it around - it is used in the while loop above, where we need to iterate through an array. To start with, each() will return the first element, then the second element, then the third, and so on, until it finds there are no elements left, in which case it will return false and end the loop.



- The meaning of that first line is “get the current element in the array, and assign its key to \$var and its value to \$val, then advance the array cursor”.
- Get both index and value of array element: If you want to read the index-name as well as its value for each array element then you can use foreach loop as follows:

```
<?php
 foreach ($array as $key => $val) {
 print "$key = $val\n";
 }
?>
```

- It is very handy when you deal with associative arrays.



# ARRAY FUNCTIONS

---

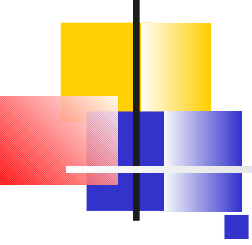
## 1. Array Shift

`array_shift` — Shift an element off the beginning of `array`

Description

Mixed `array_shift ( array &$array )`

`array_shift()` shifts the first value of the *array* off and returns it, shortening the *array* by one element and moving everything down. All numerical array keys will be modified to start counting from zero while literal keys won't be touched. If *array* is empty (or is not an array), NULL will be returned.

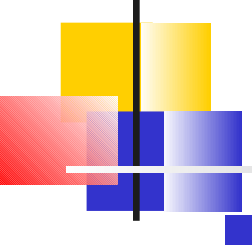


## Example 261.array\_shift() example

```
<?php
 $stack = array("orange", "banana", "apple",
 "raspberry");
 $fruit = array_shift($stack);
 print_r($stack);
?>
```

This would result in *\$stack* having 3 elements left:

```
Array
(
 [0] => banana
 [1] => apple
 [2] => raspberry
)
```



## 2.Array\_pop

(PHP 4, PHP 5)

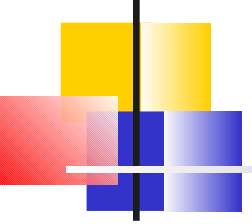
array\_pop — Pop the element off the end of arrayDescription

mixed array\_pop ( array &\$array )

array\_pop() pops and returns the last value of the *array*, shortening the *array* by one element. If *array* is empty (or is not an array), *NULL* will be returned.

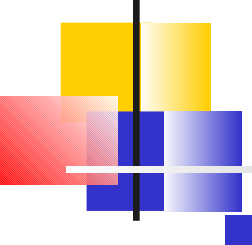
- Example:-.array\_pop() example

```
<?php
$stack = array("orange", "banana", "apple",
"raspberry");
$fruit = array_pop($stack);
print_r($stack);
?>
```

- 
- 
- After this, *\$stack* will have only 3 elements:

**Array**

```
(
 [0] => orange
 [1] => banana
 [2] => apple
) and raspberry will be assigned to $fruit.
```



### 3. Array\_push

(PHP 4, PHP 5)

`array_push` — Push one or more elements onto the end of array

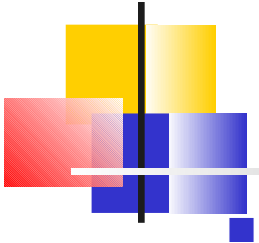
- **Description**

`int array_push ( array &$array, mixed $var [, mixed $...] )`

`array_push()` treats *array* as a stack, and pushes the passed variables onto the end of *array*. The length of *array* increases by the number of variables pushed. Has the same effect as:

```
<?php
 $array[] = $var;
?>
```

repeated for each *var*.



Example:- **array\_push()** example

```
<?php
 $stack = array("orange", "banana");
 array_push($stack, "apple", "raspberry");
 print_r($stack);
```

?>

The above example will output:

```
Array
(
 [0] => orange
 [1] => banana
 [2] => apple
 [3] => raspberry
)
```